



沈阳建筑大学

上机实验报告

课程名称:

操作系统

实验题目:

文件操作的模拟实验

专业班级:

计算机 2101 班

学 号:

2104230414

日 期:

2024.05.15

【实验目的】

1. 了解文件系统的原理；
2. 用高级语言编写和调试一个简单的文件系统，模拟文件管理的工作过程。从而对各种文件操作命令的实质内容和执行过程有比较深入的了解。

【实验准备】

文件的逻辑结构

- 顺序文件
 - 索引文件
 - 索引顺序文件
 - 直接文件和哈希文件
2. 外存分配方式
 - 连续分配
 - 链接分配
 - 索引分配

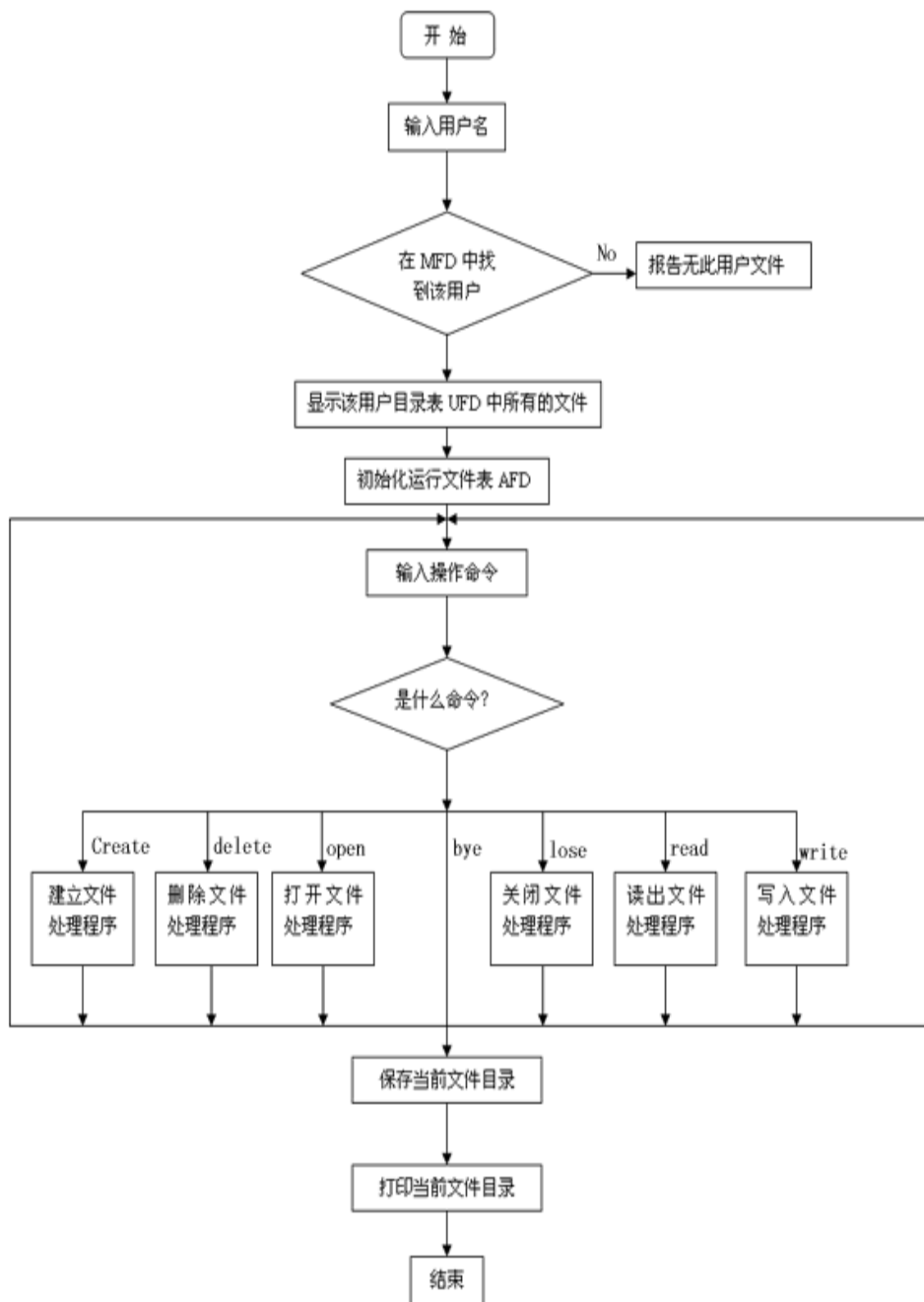
【实验内容】

1. 实验要求

要求设计一个 n 个用户的文件系统，每次用户可保存 m 个文件，用户在一次运行中只能打开一个文件，对文件必须设置保护措施，且至少有 Create、delete、open、close、read、write 等命令。

2. 实验题目

- 设计一个 10 个用户的文件系统，每次用户可保存 10 个文件，一次运行用户可以打开 5 个文件。
 - 程序采用二级文件目录（即设置主目录[MFD]）和用户文件目录（UED）。另外，为打开文件设置了运行文件目录（AFD）。
 - 为了便于实现，对文件的读写作了简化，在执行读写命令时，只需改读写指针，并不进行实际的读写操作。
3. 算法与框图



因系统小，文件目录的检索使用了简单的线性搜索。文件保护简单使用了三位保护码：允许读写执行、对应位为 1，对应位为 0，则表示不允许读写、执行。程序中使用的主要设计结构如下：主文件目录和用户文件目录（MFD、UFD）打开文件目录（AFD）（即运行文件目录）。

M D F	U F D	A F D
用户名	文件名	打开文件名
文件目录指针	保护码	打开保护码
用户名	文件长度	读写指针
文件目录指针	文件名	

【源程序代码】

```

#include<iostream>
#include<string>
#include<conio.h>
using namespace std;
struct TYPE_UFD
{
    string File_Name;
    bool Read;
    bool Write;
    bool Execute;
    int Length_File;
};
struct TYPE_MFD
{
    string User_Name;
    TYPE_UFD* Pointer;
};
struct TYPE_AFD
{
    int File_ID;
    bool Read;
    bool Write;
    bool Execute;
    int Pointer;
};
class TYPE_FILE_SYSTEM
{
public:

```

```

void Initial(void);
void Start(void);
private:
    int _Number_Users;
    int _Number_Files;
    int _MaxNumber_Open_Files;
    TYPE_MFD* _MFD;
    TYPE_UFD* _UFD;
    TYPE_AFD* _AFD;
};
void TYPE_FILE_SYSTEM::Initial(void)
{
    _Number_Users = 10;
    _Number_Files = 10;
    _MaxNumber_Open_Files = 5;
    _UFD = new TYPE_UFD[_Number_Users * _Number_Files]; _MFD = new
TYPE_MFD[_Number_Users];
    int i = 0;
    for (i = 0; i < _Number_Users; i++)
    {
        _MFD[i].Pointer = &(_UFD[i * _Number_Files]);
    }
    _AFD = new TYPE_AFD[_MaxNumber_Open_Files];
    _MFD[0].User_Name = "qc";
    _UFD[0].File_Name = "test1.txt";
    _UFD[0].Length_File = 10;
    _UFD[0].Read = true;
    _UFD[0].Write = false;
    _UFD[0].Execute = true;
    _UFD[1].File_Name = "test2.txt";
    _UFD[1].Length_File = 20;
    _UFD[1].Read = true;
    _UFD[1].Write = false;
    _UFD[1].Execute = false;
    for (i = 2; i < _Number_Files; i++)
    {
        _UFD[i].File_Name = "";
        _UFD[i].Length_File = -1;
        _UFD[i].Read = false;

```

```

        _UFD[i].Write = false;
        _UFD[i].Execute = false;
    }
}
void TYPE_FILE_SYSTEM::Start(void)
{
    int User_ID;
    int i, temp_int;
    string temp;
    char choice;
    int Number_Open_Files;
    string User_Name;
    string Command;
    TYPE_UFD* UFD;
    do
    {
        do
        {
            cout << "User name:";
            cin >> User_Name;
            for (User_ID = 0; User_ID < _Number_Users; User_ID++) {
                if (_MFD[User_ID].User_Name == User_Name)
                    break;
            }
            if (User_ID == _Number_Users)
                cout << "Bad user name , please try again ." << endl;
        } while (User_ID == _Number_Users);
        cout << "欢迎" << User_Name << "登录文件系统 !" << endl; UFD =
        _MFD[User_ID].Pointer;
        for (i = 0; i < _MaxNumber_Open_Files; i++)
        {
            _AFD[i].File_ID = -1;
        }
        Number_Open_Files = 0;
        do
        {
            cout << "C:\\\" << User_Name << ">";
            cin >> Command;
            if (Command == "dir")

```

```

{
    cout << endl;
    cout << "Files of user " << User_Name << endl;
    cout << "\t" << "State\t" << "Length\t" << "File name" << endl;
    for (i = 0; i < _Number_Files; i++) {
        if (UFD[i].Length_File != -1) {
            cout << "\t";
            if (UFD[i].Read == true)
                cout << "R";
            else
                cout << "-";
            if (UFD[i].Write == true)
                cout << "W";
            else
                cout << "-";
            if (UFD[i].Execute == true)
                cout << "E";
            else
                cout << "-";
            cout << "\t";
            cout << UFD[i].Length_File;
            cout << "\t";
            cout << UFD[i].File_Name << endl;
        }
    }
    cout << endl;
}
else if (Command == "diropen")
{
    cout << endl;
    cout << "Opening Files of user " << User_Name << endl; cout <<
"\t" << "State\t" << "Open File name" << endl; for (i = 0; i <
_MaxNumber_Open_Files; i++) {
        if (_AFD[i].File_ID != -1)
        {
            cout << "\t";
            if (_AFD[i].Read == true)
                cout << "R";
            else

```

```

        cout << "-";
        if (_AFD[i].Write == true)
            cout << "W";
        else
            cout << "-";
        if (_AFD[i].Execute == true)
            cout << "E";
        else
            cout << "-";
        cout << "\t";
        cout << UFD[_AFD[i].File_ID].File_Name << endl;
    }
}
cout << endl;
}
else if (Command == "create") {
    for (i = 0; i < _Number_Files; i++)
        if (UFD[i].Length_File == -1)
            break;
    if (i == _Number_Files)
        cout << "Error: you have already had " <<
            _Number_Files << " files ." << endl;
    else {
        cout << "请输入新文件信息:" << endl;
        cout << "文件名 : ";
        cin >> temp;
        UFD[i].File_Name = temp;
        cout << "Read (y/n):";
        do
        {
            choice = _getch();
        } while (choice != 'y' && choice != 'n');
        if (choice == 'y')
            UFD[i].Read = true;
        else
            UFD[i].Read = false;
        cout << endl;
        cout << "Write (y/n):";
        do

```



```

        {
            choice = _getch();
        } while (choice != 'y' && choice != 'n');
        if (choice == 'y')
            UFD[i].Write = true;
        else
            UFD[i].Write = false;
        cout << endl;
        cout << "Execute (y/n):";
        do
        {
            choice = _getch();
        } while (choice != 'y' && choice != 'n');
        if (choice == 'y')
            UFD[i].Execute = true;
        else
            UFD[i].Execute = false;
        cout << endl;
        cout << "Length :";
        cin >> temp_int;
        if (temp_int > 0)
            UFD[i].Length_File = temp_int;
        cout << "Ok , the new file " << UFD[i].File_Name << " is created!" <<
endl;
    }
}
else if (Command == "delete")
{
    cout << "Please enter the file name :";
    cin >> temp;
    for (i = 0; i < _Number_Files; i++)
        if ((UFD[i].Length_File != -1) && (UFD[i].File_Name ==
temp))
            break;
    if (i == _Number_Files)
        cout << "Bad file name , please try again ." << endl; else {
        UFD[i].Length_File = -1;
        cout << "Ok , the file " << UFD[i].File_Name << " is
deleted ." << endl;

```

```

    }
}
else if (Command == "open")
{
    if (Number_Open_Files == _MaxNumber_Open_Files)
        cout << "Error: you have already opened " <<
Number_Open_Files << " files." << endl;
    else
    {
        cout << "Please enter the file name :";
        cin >> temp;
        for (i = 0; i < _Number_Files; i++)
            if ((UFD[i].Length_File != -1) && (UFD[i].File_Name
== temp))

                break;
        if (i == _Number_Files)
            cout << "Bad file name , please try again ." << endl;
        else
        {
            Number_Open_Files++;
            for (temp_int = 0;
temp_int < _MaxNumber_Open_Files;
temp_int++)

                if (_AFD[temp_int].File_ID == -1)
                    break;
            _AFD[temp_int].File_ID = i;
            _AFD[temp_int].Pointer = 0;
            cout << "Please configure the open mode :" << endl;
            if (UFD[i].Read == true)
            {
                cout << "Read (y/n):";
                do
                {
                    choice = _getch();
                } while (choice != 'y' && choice != 'n');
                if (choice == 'y')
                    _AFD[temp_int].Read = true;
                else
                    _AFD[temp_int].Read = false;
            }
        }
    }
}

```

```

        cout << endl;
    }
    else
        _AFD[temp_int].Read = false;
    if (UFD[i].Write == true) {
        cout << "Write (y/n):";
        do
        {
            choice = _getch();
        } while (choice != 'y' && choice != 'n');
        if (choice == 'y')
            _AFD[temp_int].Write = true;
        else
            _AFD[temp_int].Write = false;
        cout << endl;
    }
    else
        _AFD[temp_int].Write = false;
    if (UFD[i].Execute == true)
    {
        cout << "Execute (y/n):";
        do
        {
            choice = _getch();
        } while (choice != 'y' && choice != 'n');
        if (choice == 'y')
            _AFD[temp_int].Execute = true;
        else
            _AFD[temp_int].Execute = false;
        cout << endl;
    }
    else
        _AFD[temp_int].Execute;
    cout << "Ok , the file " << temp << " is open ." << endl;
}
}
}
else if (Command == "logout")
{

```

```

        cout << "Ok , see you later , " << User_Name << " !" << endl;
        break;
    }
    else if (Command == "close")
    {
        cout << "Please enter the file name :";
        cin >> temp;
        for (i = 0; i < _Number_Files; i++)
            if ((UFD[i].Length_File != -1) && (UFD[i].File_Name ==
temp))
                break;
        if (i == _Number_Files)
            cout << "Bad file name , please try again ." << endl; else
        {
            for (temp_int = 0; temp_int < _MaxNumber_Open_Files;
temp_int++)
                if (_AFD[temp_int].File_ID == i)
                    break;
            if (temp_int == _MaxNumber_Open_Files)
                cout << "The file " << temp << " isn't open ." << endl;
            else
            {
                _AFD[temp_int].File_ID = -1;
                Number_Open_Files--;
                cout << "Ok , the file " << temp << " is closed ." <<
endl;
            }
        }
    }
    else if (Command == "read")
    {
        cout << "Please enter the file name :";
        cin >> temp;
        for (i = 0; i < _Number_Files; i++)
            if ((UFD[i].Length_File != -1) && (UFD[i].File_Name ==
temp))
                break;
        if (i == _Number_Files)
            cout << "Bad file name , please try again ." << endl; else

```

```

        {
            for (temp_int = 0; temp_int < _MaxNumber_Open_Files;
temp_int++)
                if (_AFD[temp_int].File_ID == i)
                    break;
            if (temp_int == _MaxNumber_Open_Files)
                cout << "The file " << temp << " isn't open ." << endl;
            else
            {
                if (_AFD[temp_int].Read == true)
                    cout << "Ok , read the file " << temp << "
successfully." << endl;
                else
                    cout << "Error: the open mode of the file doesn't
allow you to read it ." << endl;
            }
        }
    }
    else if (Command == "write")
    {
        cout << "Please enter the file name :";
        cin >> temp;
        for (i = 0; i < _Number_Files; i++)
            if ((UFD[i].Length_File != -1) && (UFD[i].File_Name ==
temp))
                break;
        if (i == _Number_Files)
            cout << "Bad file name , please try again ." << endl; else
        {
            for (temp_int = 0; temp_int < _MaxNumber_Open_Files;
temp_int++)
                if (_AFD[temp_int].File_ID == i)
                    break;
            if (temp_int == _MaxNumber_Open_Files)
                cout << "The file " << temp << " isn't open ." << endl;
            else
            {
                if (_AFD[temp_int].Write == true)
                    cout << "Ok , write the file " << temp << "

```

```

successfully." << endl;

                                else
                                    cout << "Error: the open mode of the file doesn't
allow you to write it ." << endl;
                                }
                            }
                        }
                    else if (Command == "shutdown")
                    {
                        cout << "Logging out....." << endl;
                        cout << "Ok , see you later , " << User_Name << " !" << endl;
                        cout << "The power of the computer is shutting down....." <<
endl;

                        break;
                    }
                else
                {
                    cout << "Bad commands , please try again ." << endl;
                }
            } while (Command != "logout" && Command != "shutdown");
        } while (Command != "shutdown");
    }
}

int main() {
    TYPE_FILE_SYSTEM FS;
    FS.Initial();
    FS.Start();
    return 0;
}

```

运行结果截图:

```

E:\操作系统\实验4_文件操作\
User name:qc
欢迎qc登录文件系统 !
C:\qc>dir

Files of user qc
      State   Length  File name
      R-E      10    test1.txt
      R--      20    test2.txt

C:\qc>create
请输入新文件信息:
文件名 : zqc
Read (y/n):
Write (y/n):
Execute (y/n):
Length :100
Ok , the new file zqc is created!
C:\qc>dir

Files of user qc
      State   Length  File name
      R-E      10    test1.txt
      R--      20    test2.txt
      RW-     100    zqc

```

```
C:\qc>diropen
Opening Files of user qc
State   Open File name

C:\qc>open
Please enter the file name :zqc
Please configure the open mode :
Read (y/n):
Write (y/n):
Ok , the file zqc is open .
C:\qc>
```

【实验体会】

这是操作系统的最后一次上机实验，我认为，文件系统的管理才是让我们普通用户感受操作系统最核心的部分，那种在控制台，一条条输入指定来调用相应的功能，完成文件的创建、读写、删除等操作，有点类似与 Linux 的命令行窗口。如果以后能有几乎开发一个自己的操作系统，那肯定会很有成就感。

【思考题】

什么是逻辑文件，什么是物理文件，它们有什么区别？

逻辑文件是用户视角下的文件，它表示了文件的组织方式和逻辑结构，比如记录的顺序和字段的格式。用户通过文件名和目录结构来访问逻辑文件，而无需关心文件在磁盘上的具体布局。

物理文件则是指文件在实际存储介质上的存储形式，涉及文件在磁盘上的具体位置、存储空间分配和访问方法等细节。物理文件通常由操作系统和文件系统管理，对用户透明。

简而言之，逻辑文件关注文件的内容和结构，而物理文件关注文件在存储设备上的实际表示，两者在视图角度、存储表示、管理方式和访问方式等方面存在区别。